Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Гуманитарный институт
Кафедра информационных технологий в креативных и культурных индустриях

РЕФЕРАТ
по дисциплине «Информационные системы и технологии»:
Технологии создания и обработки табличных данных

Руководитель		ст. преподаватель И. Р. Нигматуллин
Студент	ГФ26-02Б, 152650266	В. А. Новикова
Студент	ГФ26-02Б, 152650648	К. М. Рыженкова
Студент	ГФ26-02Б, 152651218	А. Савельева
Студент	ГФ26-02Б, 152646583	Е. Е. Стрельская

Красноярск 2026

ВВЕДЕНИЕ

Табличные данные — одна из базовых форм представления информации, с которой сталкивается и исследователь, и аналитик, и специалист в креативных индустриях. Электронные таблицы позволяют быстро ввести данные, выполнить расчёт и построить диаграмму, однако за кажущейся простотой скрывается целый набор требований к структуре, форматам хранения и контролю ошибок, от которых зависит корректность любых выводов.

Цель настоящей работы — рассмотреть технологии создания и обработки табличных данных. Для достижения цели поставлены следующие задачи:

– определить принципы организации табличных данных для машинной обработки;

– сравнить форматы хранения и обмена и оценить риски потерь при конвертации;

– рассмотреть формы визуализации и проблему искажений;

– описать автоматизацию расчётов формулами и контроль ошибок;

– раскрыть этапы очистки и подготовки данных к анализу;

– определить возможности и пределы применимости электронных таблиц в учебной и исследовательской работе.

Объект исследования — табличные данные как форма представления информации. Предмет — технологии их создания, хранения, обработки и визуализации.

1 СТРУКТУРА ТАБЛИЧНЫХ ДАННЫХ: ПРИНЦИПЫ ОРГАНИЗАЦИИ ДЛЯ МАШИННОЙ ОБРАБОТКИ

Табличные данные, предназначенные для машинной обработки, принципиально отличаются от таблиц, созданных исключительно для визуального восприятия человеком. Если человек легко прощает объединённые ячейки, многоуровневые заголовки и цветовое кодирование, то алгоритм требует строгой регулярности: каждая строка должна представлять одно наблюдение, каждый столбец — одну переменную с фиксированным типом данных.

Базовый принцип организации — так называемая «аккуратная» (tidy) структура, при которой таблица имеет плоскую иерархию: одна строка заголовка, отсутствие пропусков в ключевых полях, единый формат для каждого столбца. Нарушение этого принципа — например, когда в одной колонке смешаны числа и текст или когда итоговая строка встроена в тело таблицы — приводит к тому, что при автоматическом чтении тип столбца определяется неверно, а агрегатные операции дают искажённый результат.

С точки зрения машинной обработки критично и то, что таблица должна быть самоописываемой: наличие заголовков, явное указание единиц измерения, отсутствие скрытых листов и внешних связей. Именно поэтому при подготовке данных к анализу рекомендуется предварительно «нормализовать» таблицу: привести имена столбцов к единому регистру и стилю, удалить пустые строки и столбцы, отделить заголовки от данных. Библиотеки подготовки данных, такие как Datra или tidytable, реализуют эти операции автоматически, подчёркивая, что качественная структура — это не эстетическое требование, а условие корректности последующих вычислений [4].

Таким образом, структура табличных данных для машинной обработки строится на трёх принципах: атомарность наблюдений, типовая однородность столбцов и отсутствие встроенной в тело таблицы служебной информации.

2 ФОРМАТЫ ХРАНЕНИЯ И ОБМЕНА ТАБЛИЧНЫМИ ДАННЫМИ: ПОТЕРИ ПРИ КОНВЕРТАЦИИ

Выбор формата хранения и обмена — это всегда компромисс между выразительностью и переносимостью. Наиболее распространённые форматы — XLSX, CSV, JSON и XML — обладают разной устойчивостью к потерям при конвертации.

Критический этап — экспорт из Excel в CSV. Excel хранит не только значения, но и формулы, форматирование, комментарии, проверки данных и типы. CSV сохраняет только «голые» строки, и вся семантическая нагрузка теряется безвозвратно. Более того, русскоязычная версия Excel по умолчанию использует кодировку Windows-1251 и разделитель «точка с запятой», что расходится с международным стандартом RFC 4180 (запятая, UTF-8) [1]. При чтении такого файла в Python или на Linux-сервере кириллица превращается в нечитаемые символы, а структура колонок нарушается.

CSV также крайне чувствителен к синтаксическим повреждениям: пропущенная кавычка в поле с запятой внутри сдвигает все последующие колонки, и восстановить исходное выравнивание без ручной проверки невозможно. JSON, напротив, имеет строгий синтаксис, но любой пропуск скобки или запятой делает невалидным весь документ — частичное чтение невозможно. Excel-файлы устойчивее к частичным повреждениям за счёт бинарной структуры, но их сложнее обрабатывать программно без специализированных библиотек.

Отдельный класс рисков — потеря метаданных: определения пропущенных значений, форматы дат, точность чисел с плавающей точкой. При конвертации между статистическими пакетами и текстовыми форматами метки переменных и значений могут исчезнуть или усечься. Практическая рекомендация: для обмена использовать CSV с явным указанием кодировки UTF-8 и разделителя, для долгосрочного хранения — открытые форматы с сохранением схемы (например, Parquet или JSON с метаописанием), а конвертацию всегда сопровождать проверкой контрольных сумм и выборочным сравнением до и после.

3 ВИЗУАЛИЗАЦИЯ ТАБЛИЧНЫХ ДАННЫХ: ВЫБОР ФОРМЫ ПРЕДСТАВЛЕНИЯ И ПРОБЛЕМА ИСКАЖЕНИЙ

Визуализация табличных данных решает две противоположные задачи: сделать закономерности видимыми и не солгать при этом. Выбор формы представления зависит от типа данных и вопроса, на который должна ответить визуализация. Для категориальных данных с малым числом категорий подходит столбчатая диаграмма; для распределения — гистограмма или ящик с усами; для связи двух количественных переменных — точечная диаграмма; для временных рядов — линейный график.

Проблема искажений возникает на нескольких уровнях. Первый — масштаб оси. Обрезанная ось Y (например, начинающаяся не с нуля, а с 95% максимального значения) визуально преувеличивает незначительные различия. Второй — агрегация. Суммирование или усреднение по слишком крупным группам может скрыть выбросы и создать иллюзию однородности. Третий — перегруженность. Попытка отобразить все переменные сразу приводит к «спагетти-графикам», где закономерности не читаются.

В учебной и исследовательской работе особенно важно различать исследовательскую визуализацию (для себя, для поиска гипотез) и презентационную (для читателя, для доказательства). В первом случае допустимы быстрые, несовершенные графики; во втором требуется сознательный контроль искажений: подписи осей с единицами, указание размера выборки, отказ от трёхмерных эффектов и «нарядных» цветов в пользу различимости. Электронные таблицы предоставляют базовый набор диаграмм, но для серьёзной визуализации табличных данных чаще используют специализированные библиотеки (matplotlib, ggplot2, seaborn), где контроль над осями, масштабом и аннотациями полнее.

ФОРМУЛЫ И ФУНКЦИИ ЭЛЕКТРОННЫХ ТАБЛИЦ: АВТОМАТИЗАЦИЯ РАСЧЁТОВ И КОНТРОЛЬ ОШИБОК

Формулы и функции — главный инструмент автоматизации в электронных таблицах, но именно они являются источником наиболее коварных ошибок, поскольку ошибка в одной ячейке может каскадно распространиться на весь отчёт.

Типичные категории ошибок в Excel хорошо документированы: `#DIV/0!` при делении на ноль или пустую ячейку; `#N/A` при неудачном поиске (например, в ВПР/VLOOKUP); `#NAME?` при опечатке в имени функции или неверной ссылке на лист; `#REF!` при удалении ячейки, на которую ссылалась формула; `#VALUE!` при попытке арифметики с текстом [3]. Особый класс — логические ошибки, не проявляющиеся как код ошибки: ссылка на пустую ячейку в среднем значении занижает результат, а «зашитая» в формулу константа вместо ссылки на ячейку делает расчёт негибким и трудно проверяемым.

Контроль ошибок строится на нескольких практиках. Во-первых, использование встроенного механизма проверки ошибок Excel (Formulas → Error Checking), который находит несогласованные формулы в соседних ячейках и ссылки на пустые ячейки [2]. Во-вторых, явная обработка исключений: `ЕСЛИОШИБКА` / `IFERROR` позволяет заменить код ошибки на осмысленное значение или ноль, но требует осторожности, чтобы не скрыть реальную проблему. В-третьих, дисциплина ссылок: абсолютные ссылки (`$A$1`) для констант, именованные диапазоны для повторяющихся областей, отказ от магических чисел. В-четвёртых, проверка типов: если ячейка отформатирована как «Текст», формула может отображаться, а не вычисляться — распространённая причина «не работающих» формул [3].

Для учебных работ полезно правило: любая формула сложнее суммы или среднего должна быть проверена на тестовом наборе с известным ответом. Автоматизация без контроля ошибок создаёт иллюзию точности, которая разрушается при первом же изменении исходных данных.

4 ОЧИСТКА И ПОДГОТОВКА ТАБЛИЧНЫХ ДАННЫХ К АНАЛИЗУ

Очистка данных — этап, который в реальных проектах занимает больше времени, чем сам анализ, и пренебрежение им обесценивает любые последующие выводы. Подготовка включает несколько последовательных слоёв работы.

Первый слой — структурная очистка: удаление полностью пустых строк и столбцов, устранение объединённых ячеек, приведение заголовков к машиночитаемому виду (без пробелов, специальных символов, с единым регистром). Библиотеки подготовки данных автоматизируют это: например, `clean_variable_names()` преобразует имена в snake_case и даже распознаёт серийные даты Excel, превращая число 44197 в корректную дату [5].

Второй слой — типовая очистка. Числа, сохранённые как текст (часто после экспорта из CSV), даты в неоднозначных форматах, логические значения в виде «да/нет» или «1/0» — всё это требует явного приведения типов. Инструменты вроде `infer_data_types()` анализируют столбец и автоматически преобразуют его, если доля валидных значений превышает порог [4]. Отдельная проблема — «призраки формул»: значения `#DIV/0!`, `#N/A`, `#VALUE!`, попавшие в выгрузку, должны быть заменены на пропуски (NA) до любого анализа, иначе они либо вызовут ошибку, либо будут молча проигнорированы с искажением статистики [3].

Третий слой — содержательная очистка: обработка пропущенных значений, дубликатов и выбросов. Здесь нет универсального рецепта: удаление строк с пропусками допустимо при малой доле пропусков и случайном характере их возникновения; заполнение медианой или модой — компромисс, который нужно явно документировать. Дубликаты могут быть полными (одинаковые строки) или частичными (одинаковый идентификатор с разными датами) — во втором случае правило разрешения конфликта должно быть определено до очистки [4].

Ключевой принцип: очистка должна быть воспроизводимой и документированной. Каждое преобразование — замена пропусков, удаление выбросов, переименование категорий — фиксируется в скрипте или отчёте, чтобы результат можно было проверить и повторить.

5 ЭЛЕКТРОННЫЕ ТАБЛИЦЫ В УЧЕБНОЙ И ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ: ВОЗМОЖНОСТИ И ПРЕДЕЛЫ ПРИМЕНИМОСТИ

Электронные таблицы занимают уникальную нишу в учебной и исследовательской работе благодаря низкому порогу входа и мгновенной обратной связи. Студент, впервые сталкивающийся с данными, может ввести числа, построить диаграмму и увидеть результат за минуты, не изучая синтаксис языка программирования. Для небольших наборов данных (сотни, иногда тысячи строк) Excel или LibreOffice Calc остаются адекватным инструментом разведочного анализа, построения сводных таблиц и подготовки иллюстраций.

Однако пределы применимости начинаются там, где заканчивается «небольшой набор». Электронные таблицы плохо масштабируются: при десятках тысяч строк производительность падает, а вероятность случайного изменения ячейки возрастает. Отсутствие версионного контроля в привычном смысле (история изменений в Excel есть, но она не заменяет Git) делает совместную работу рискованной: две версии файла с одинаковым именем могут содержать разные данные. Формулы, разбросанные по ячейкам, труднее проверить и воспроизвести, чем скрипт: в скрипте виден весь конвейер обработки, в таблице — только конечный результат.

Для исследовательской работы критично и то, что электронные таблицы поощряют неявные решения. Сортировка, выполненная мышью, не оставляет следа; фильтр, скрывший часть строк, может быть забыт; «подчистка» выбросов вручную недокументируема. Воспроизводимость — базовое требование научного исследования — страдает.

Разумная стратегия: использовать электронные таблицы на этапе сбора, первичного осмотра и визуализации, но переносить обработку в скрипты (Python, R) по мере роста сложности. Таблицы хороши как блокнот и как средство коммуникации с неспециалистами; они плохи как среда для воспроизводимого анализа. Понимание этой границы — часть цифровой грамотности, которую и должен формировать учебный курс.

ЗАКЛЮЧЕНИЕ

В работе рассмотрены технологии создания и обработки табличных данных. Показано, что структура таблицы для машинной обработки строится на атомарности наблюдений, типовой однородности столбцов и отсутствии служебной информации в теле таблицы. Сравнение форматов хранения и обмена выявило, что экспорт в CSV сопряжён с потерями семантики и рисками кодировки, а конвертацию необходимо сопровождать проверкой. Рассмотрены формы визуализации и источники искажений, практики контроля ошибок формул, а также трёхслойная модель очистки данных — структурная, типовая и содержательная. Сделан вывод о разумной стратегии: электронные таблицы эффективны на этапе сбора, первичного осмотра и визуализации, тогда как воспроизводимый анализ требует переноса обработки в скрипты.

Поставленные задачи выполнены, цель работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Shafranovich Y. Common Format and MIME Type for Comma-Separated Values (CSV) Files. RFC 4180, IETF, 2005. URL: https://www.rfc-editor.org/rfc/rfc4180.html (дата обращения: 08.11.2026).
2. Microsoft Support.(Fix an inconsistent formula). URL: https://api.support.office.com/zh-cn/article-5dd940a1-4f87-44bd-91dd-bf45ed828f05 (дата обращения: 08.11.2026).
3. Microsoft Support. Detect formula errors in Excel. URL: https://support.microsoft.com/ar-sa/Excel/detect-formula-errors-in-excel (дата обращения: 08.11.2026).
4. PyPI. datra 0.1.0. URL: https://pypi.org/project/datra/ (дата обращения: 08.11.2026).
5. Fairbanks M. Package {tidytable}. CRAN. URL: https://cran.r-project.org/web/packages/tidytable/DESCRIPTION (дата обращения: 08.11.2026).